



DOSSIER DE PROJET

OpsForge

Console locale de gestion d'incidents et chaîne DevOps de bout en bout

Titre professionnel **Administrateur système DevOps** — niveau 6

Code titre TP-01414 · RNCP 36061

Candidat **Dyllan Thouvignon**

Organisme de formation **Liora (ex DataScientest)**

Session d'examen **7 septembre 2026 — 09h30**

Campus Omnes Cœur Défense II, Courbevoie

Type de projet **Projet fil rouge indépendant, cahier des charges conçu par le candidat**

Dépôt Git **ThDyllan/opsforge**

Candidat technique gelé **branche phase6-operator-ux — commit a9ec694**

Version finale — août 2026

Sommaire

Le projet en 60 secondes.....	2
Introduction.....	2
1. Compétences du référentiel couvertes par le projet	3
1.1 Les trois compétences obligatoires, critère par critère	3
1.2 Compétences partiellement mises en pratique.....	4
2. Cahier des charges	5
2.1 Contexte	5
2.2 Problématique et objectifs	5
2.3 Utilisateur et cas d'usage.....	5
2.4 Besoins.....	6
2.5 Contraintes	7
2.6 Livrables et critères d'acceptation.....	7
2.7 Hors périmètre et évolution du périmètre	7
3. Spécifications techniques	8
3.1 Architecture logique	8
3.2 Topologie de déploiement.....	9
3.3 Le domaine applicatif.....	9
3.4 Conteneurs : de Compose à Kubernetes	11
3.5 Stockage et persistance (prouvée)	11
3.6 Supervision : réel contre simulé	12
3.7 Sauvegarde et restauration	14
3.8 Sécurité	14
3.9 Environnements et versions	14
4. Démarche de travail et outils	15
4.1 Le découpage en phases et leur état de validation	15
4.2 Workflow Git.....	15
4.3 Outils.....	15
4.4 Conduite du projet et usage de l'IA	15
5. Réalisations significatives	16
5.1 Instrumentation et alerte de supervision (CP n° 10)	16
5.2 Image Docker durcie (CP n° 7)	16
5.3 Workloads Kubernetes durcis et stockage persistant (CP n° 7)	17
5.4 Pipeline d'intégration continue (chaîne 1)	17
5.5 Automatisation du déploiement d'infrastructure avec Ansible (CP n° 2, chaîne 2)	18
5.6 Sauvegarde et restauration sûres.....	20
6. Situation de travail ayant nécessité une recherche	21
Le fichier ansible.cfg silencieusement ignoré dans le control node conteneurisé	21
7. Synthèse des validations	22
Clôture de la phase 6 : ce que la revue humaine a trouvé	23
8. Limites assumées et évolutions.....	24
Conclusion	25
Annexe A — Chronologie détaillée du projet.....	26
Annexe B — Inventaire des preuves	27
B.1 Captures et schémas (versionnés sous deliverables/assets/).....	27
B.2 Journaux d'exécution bruts (versionnés sous deliverables/evidence/).....	27
Annexe C — Reproduire les preuves	28
C.1 Application et tests	28
C.2 Déploiement de l'infrastructure par Ansible (CP n° 2)	28
C.3 Persistance du stockage (CP n° 7).....	28
C.4 Supervision et déclenchement réel de l'alerte (CP n° 10)	28
C.5 Sauvegarde et restauration	28
Annexe D — Glossaire	29

Le projet en 60 secondes

OpsForge est une console locale de gestion d'incidents : un opérateur y qualifie des signaux (alertes), ouvre et traite des incidents, applique des procédures contrôlées (runbooks) et conserve la preuve de chaque action dans un journal d'audit. Autour de cette application, j'ai construit et validé une chaîne DevOps locale complète : conteneurisation, intégration continue, sauvegarde/restauration, déploiement Kubernetes, supervision, et automatisation du déploiement de l'infrastructure.

Les **trois compétences obligatoires** du titre sont couvertes, chacune avec une preuve distincte :

Compétence obligatoire	Preuve principale	Où
Automatiser le déploiement d'une infrastructure	Playbook Ansible : toute l'infrastructure locale (cluster k3d, PostgreSQL, API, supervision) déployée et vérifiée en une commande idempotente — <code>/health</code> et <code>/ready</code> en 200, sinon échec	§ 5.5 § 6
Gérer des containers	Image durcie non-root, orchestration Compose puis Kubernetes (securityContext complet, probes distinctes), stockage persistant dont la persistance est prouvée après destruction du pod	§ 5.2 § 5.3 § 3.5
Exploiter une solution de supervision	Application instrumentée, Prometheus qui la scrape, dashboard Grafana, et une alerte réelle (OpsForgeApiDown) observée <code>inactive</code> → <code>pending</code> → <code>firing</code> → résolue lors d'une panne provoquée	§ 5.1 § 3.6

Le périmètre est assumé : tout est local et démontrable ; il n'y a ni cloud, ni registre d'images, ni déploiement continu distant, ni authentification — ce sont des choix documentés, pas des oublis (§ 8).

Introduction

Je m'appelle Dyllan Thouvignon. Mon parcours était initialement orienté développement (BTS SIO option SLAM), puis j'ai suivi une formation DevOps en alternance. Mon activité en entreprise étant restée principalement orientée support technique, elle ne m'a pas fourni un projet DevOps suffisamment complet pour couvrir les attendus certificatifs.

À l'issue de ma formation, j'ai donc conçu et réalisé **OpsForge**, un projet fil rouge indépendant, construit de juin à août 2026 pour préparer la certification. Ce n'est pas un projet de mon entreprise d'alternance : il n'utilise aucune donnée ni infrastructure professionnelle, et j'en ai défini le cahier des charges moi-même, à partir des compétences et des attendus du référentiel.

Le fil conducteur du projet : chaque brique doit être **réellement démontrée** — tests exécutés, alerte réellement déclenchée, persistance réellement prouvée, déploiement réellement rejoué — et chaque limite est écrite noir sur blanc dans le dépôt. Ce dossier suit ce principe : il distingue systématiquement ce qui a été exécuté et vérifié de ce qui a seulement été conçu.

1. Compétences du référentiel couvertes par le projet

1.1 Les trois compétences obligatoires, critère par critère

Le REAC (Référentiel Emploi Activités Compétences ; « CP » = compétence professionnelle) définit des critères de performance précis. Voici la confrontation exacte d'OpsForge à ces critères.

CP n° 2 — Automatiser le déploiement d'une infrastructure

Critère REAC	Réponse OpsForge
« Les serveurs déployés sont fonctionnels »	Le rôle Ansible <code>verify</code> échoue si un pod API n'est pas Running ou si <code>/health</code> et <code>/ready</code> ne répondent pas HTTP 200 (<code>/ready</code> exécute un <code>SELECT 1</code> sur PostgreSQL). Chaque étage est appliqué avec <code>wait: true</code> . Preuve rejouée le 12/08/2026 : <code>ok=22 changed=9 failed=0, /health → 200, /ready → 200</code>
« L'architecture est conforme au cahier des charges »	Le playbook orchestre les manifests versionnés de k8s/ (namespaces, StatefulSet PostgreSQL + PVC, Deployment API + NodePort, Prometheus + Grafana) — il déploie l'architecture documentée, sans rien redéfinir
« Les scripts sont documentés »	Cinq rôles courts et commentés, variables centralisées, <code>ansible/README.md</code> , justification et correspondance RNCP dans <code>docs/ANSIBLE.md</code> , décision d'architecture ADR 029
Savoir associé : « outil de type Ansible ou Terraform »	Ansible + collection <code>kubernetes.core</code> (choix argumenté en § 5.5)

Limite : cible **k3d local**, pas un fournisseur cloud — l'activité-type « ...dans le cloud » n'est pas revendiquée (la CP n° 4 relèvera de l'entretien technique).

CP n° 7 — Gérer des containers

Critère REAC	Réponse OpsForge
« Les containers sont opérationnels »	Compose : API + PostgreSQL avec healthchecks, démarrage de l'API conditionné à la santé de la base. Kubernetes : pods 1/1 Running avec probes liveness/readiness distinctes (état constaté le 12/08/2026, § 3.5)
« Les containers sont connectés au réseau »	Réseau Compose interne (db : 5432) ; Services Kubernetes ClusterIP (PostgreSQL interne uniquement) et NodePort 30080 exposé sur 127.0.0.1:8080
« Les containers sont connectés au stockage distant »	PostgreSQL est connecté à un stockage persistant externalisé du cycle de vie du conteneur : volume nommé en Compose, PVC de 1 Gi monté par le StatefulSet en Kubernetes — la connexion du conteneur à un stockage porté par le système hôte figure parmi les savoir-faire associés à cette compétence dans le REAC. La persistance est prouvée : une donnée survit à la destruction puis à la recréation du pod (§ 3.5). <i>Limite assumée</i> : dans l'environnement k3d utilisé, la <code>StorageClass Local-path</code> reste locale au nœud — ce n'est pas un stockage réseau ni distribué ; une <code>StorageClass</code> réseau (NFS, CSI) la remplacerait sans modifier le manifest (§ 8).
« Les containers sont mis à jour »	Cycle explicite : rebuild de l'image (tags <code>phase4</code> → <code>phase5</code> → <code>phase6</code>), <code>k3d image import</code> , rollout du Deployment ; <code>imagePullPolicy</code> : <code>Never</code> rend visible l'absence volontaire de registre

CP n° 10 — Exploiter une solution de supervision

Critère REAC	Réponse OpsForge
« Les indicateurs définis sont pertinents »	Métriques applicatives réelles (opsforge_http_requests_total, opsforge_http_request_duration_seconds, labels méthode / route template / code) et métrique up du scrape ; dashboard : disponibilité, volume, répartition par code, latence p95, répartition par route
« Les alertes sont correctement interprétées »	Règle OpsForgeApiDown (up{job="opsforge-api"} == 0 pendant 30 s). Panne provoquée, cycle complet observé et journalisé : inactive → pending → firing → restauration → inactive (validé en phase 5, rejoué en direct le 12/08/2026 — § 3.6)
« Les échanges avec les développeurs sont réguliers »	<i>Limite du contexte individuel, assumée</i> : il n'y a pas d'équipe de développement — je tiens les deux rôles. La boucle supervision → développement existe réellement et elle est tracée dans les décisions du dépôt : ajout de /ready (ADR 022), labels de route en template pour maîtriser la cardinalité (ADR 017), configuration de scrape statique (ADR 018). En contexte d'équipe, ces constats seraient précisément le contenu des échanges avec les développeurs

1.2 Compétences partiellement mises en pratique

Je ne revendique pas ces compétences comme couvertes ; le projet en met en œuvre une partie, exploitable lors des échanges avec le jury :

CP	Mis en pratique	Ce qui manque
CP 1 — Créer des serveurs par scripts	Création scriptée et idempotente du nœud k3d ; scripts PowerShell et bash	Pas de création de VM serveur au sens strict
CP 3 — Sécuriser l'infrastructure	Non-root, rootfs en lecture seule, capabilities supprimées, seccomp, secrets hors Git, scan Trivy, bind loopback	Pas de référentiel ANSSI formalisé, ni pare-feu, ni TLS, ni authentification
CP 5 — Environnement de test	Base PostgreSQL éphémère par test, cluster k3d jetable, conteneur de service en CI	Pas d'environnement mis à disposition d'une équipe
CP 6 — Stockage des données	PostgreSQL sur deux environnements, PVC prouvé, sauvegarde et restauration testées	Pas de réplication ni de gestion formalisée des droits
CP 8 — Mise en production avec une plateforme	Déploiement Kubernetes automatisé par Ansible, vérifié de bout en bout	Pas de pré-production et production distinctes, pas de CD
CP 9 — Statistiques de services	Indicateurs choisis et justifiés (ADR 017)	Pas de SLA formalisés

Non couvertes. La CP n° 4 (mise en production dans le cloud) : aucun cloud n'a été déployé ; conformément aux modalités, elle relèvera du questionnaire complémentaire de l'entretien technique. La CP n° 11 (échanger sur des réseaux professionnels, éventuellement en anglais) : elle est évaluée par une épreuve dédiée, le questionnaire professionnel en anglais — je note simplement que la documentation du dépôt et les messages de commit sont rédigés en anglais.

2. Cahier des charges

2.1 Contexte

OpsForge est un projet fil rouge indépendant, réalisé dans le cadre de ma préparation au titre. J'en ai conçu le cahier des charges à partir du référentiel. Le domaine choisi — la gestion d'incidents — est inspiré de mon expérience de support : je connais le cycle « signal → prise en charge → procédure → traçabilité » pour l'avoir vécu au quotidien, ce qui m'a permis de définir un domaine métier crédible sans copier un outil existant ni utiliser la moindre donnée professionnelle réelle.

2.2 Problématique et objectifs

Problématique. Quand un service supervisé se dégrade, comment garantir qu'un opérateur puisse qualifier le signal, décider d'une prise en charge, appliquer une procédure sûre et prouver ensuite chaque décision — et comment livrer cette application avec une vraie chaîne DevOps : tests automatisés, conteneurs, déploiement automatisé, supervision réelle, sauvegardes vérifiées ?

Objectifs.

- Une application métier fonctionnelle et démontrable, au cycle de vie strict et à l'audit systématique.
- Une chaîne DevOps locale couvrant les trois compétences obligatoires du titre.
- Une **preuve d'exécution réelle** pour chaque brique — pas seulement du code.
- Un ensemble simple, explicable et défendable à l'oral, aux limites connues et écrites.

2.3 Utilisateur et cas d'usage

L'utilisateur est un **opérateur unique** (rôle : technicien d'exploitation). Parcours type, entièrement réalisable dans l'interface :

- Un signal arrive sur un service du catalogue → il est enregistré comme **alerte** (new).
- L'opérateur **acquitte** l'alerte, puis décide s'il ouvre un **incident** (une alerte n'a qu'un incident actif à la fois).
- Il s'attribue l'incident, le passe en **investigation**, applique un **runbook** — checklist manuelle ou automatisation limitée à une liste d'actions approuvées dans le code.
- Chaque action alimente le **journal d'audit** et la timeline de l'incident.
- Il **résout** l'incident, puis résout l'alerte séparément : le cycle du signal et celui de la prise en charge sont indépendants.

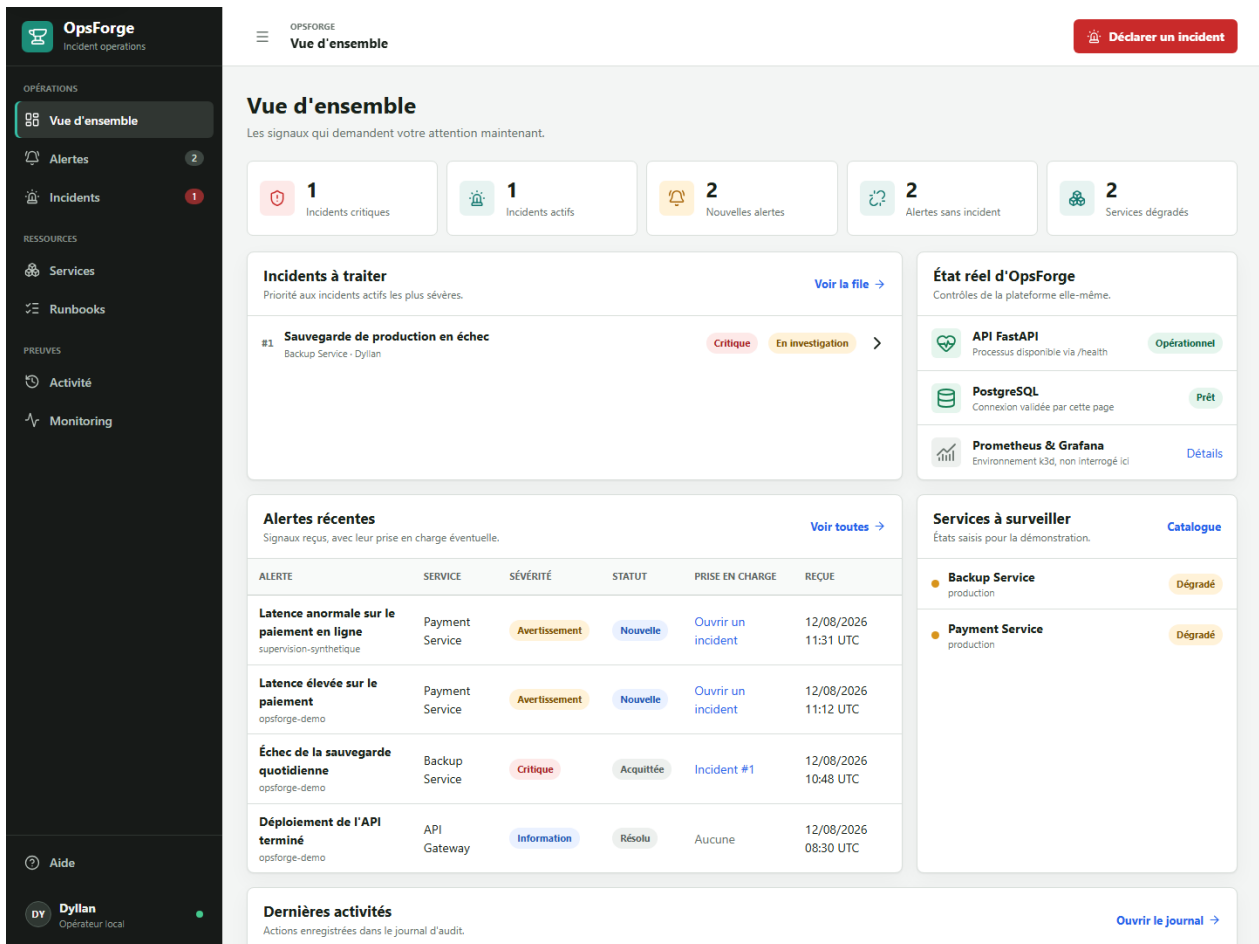


Figure 1 — Vue d'ensemble de la console au démarrage d'une prise de poste : incidents à traiter, alertes récentes avec leur état, état réel de la plateforme (à droite, « contrôles de la plateforme elle-même ») et services de démonstration.

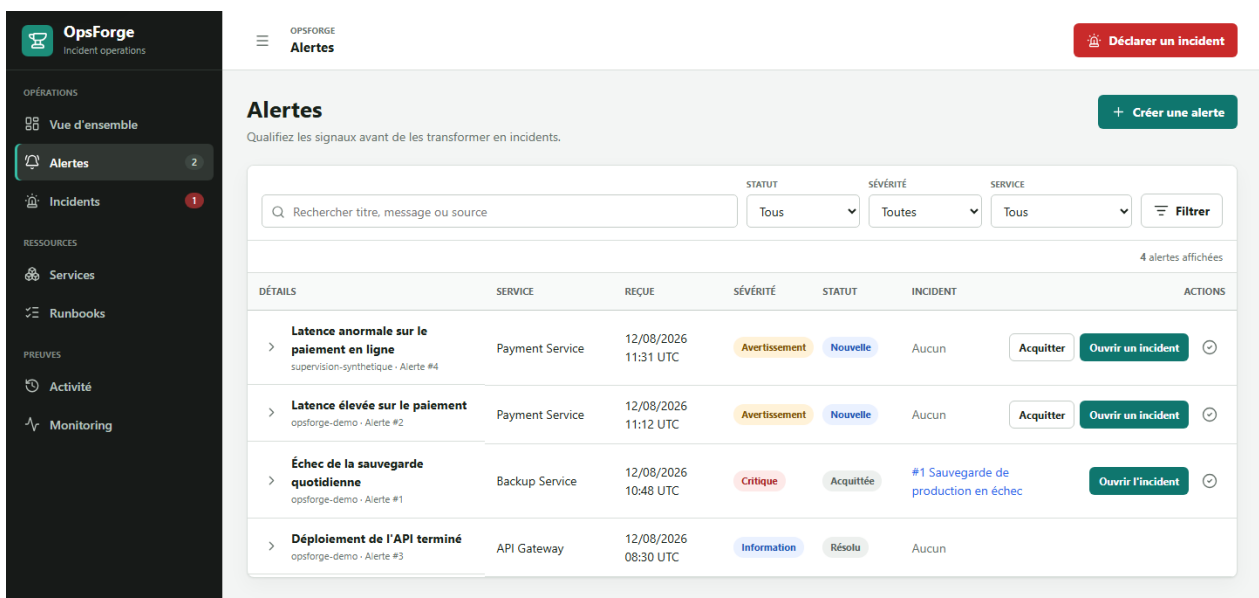


Figure 2 — La file d'alertes, premier écran du parcours : les trois états (Nouvelle, Acquittée, Résolu), les filtres, et le lien vers l'incident déjà ouvert sur l'alerte critique — une alerte n'ayant qu'un incident actif, le bouton « Ouvrir un incident » y est remplacé par « Ouvrir l'incident ».

2.4 Besoins

Fonctionnels. Catalogue de services ; file d'alertes (cycle new → acknowledged → resolved, strictement en avant) ; incidents (cycle open → investigating → resolved, un seul incident actif par alerte, incident résolu en lecture seule) ; runbooks manuels à checklist et automatisations sur liste approuvée, jamais de commande

arbitraire ; audit de toutes les mutations, y compris les échecs contrôlés ; console multipage utilisable sans outil externe ; distinction affichée entre contrôles réels de la plateforme et états métier simulés.

Techniques. API FastAPI + PostgreSQL ; image Docker non-root avec healthcheck ; environnement Docker Compose complet ; CI à chaque push (lint, tests SQLite, test d'intégration PostgreSQL, build, scan de vulnérabilités) ; sauvegarde `pg_dump` scriptée avec restauration de vérification sans risque ; déploiement Kubernetes local (Deployment, StatefulSet, ConfigMap, Secret, PVC, probes, NodePort) ; supervision Prometheus/Grafana avec une alerte démontrable ; **déploiement complet de l'infrastructure en une commande idempotente** (besoin ajouté en cours de projet — § 2.7) ; traçabilité générale (Git, documentation par phase, décisions d'architecture).

2.5 Contraintes

- Poste **Windows 11 + Docker Desktop** : ce choix contraint directement l'outillage (k3d plutôt qu'une VM, PowerShell pour les sauvegardes, control node Ansible conteneurisé).
- **Projet individuel** mené en parallèle de mon activité professionnelle, sur une période resserrée (juin → août 2026) : phases courtes, finies et validées une à une.
- **Aucune donnée réelle**, budget **zéro cloud**, et **aucune exécution de commande arbitraire** par l'application (vérifié par inspection et par test).
- Changement de poste de travail en cours de projet : la reproductibilité (Git, images, manifests) devait le permettre — et l'a permis.

2.6 Livrables et critères d'acceptation

Livrables. Le dépôt Git complet (application, tests, Dockerfile, Compose, k8s/, `ansible/`, scripts, CI) ; la documentation projet (`docs/` : architecture, guides, 29 décisions d'architecture, risques, un fichier de vérification daté par phase) ; le présent dossier et son support de présentation.

Critères d'acceptation globaux. Environnement Compose fonctionnel (`/health`, `/ready`, console) ; tests verts en local et en CI ; pods Kubernetes 1/1 Ready et application joignable depuis Windows ; persistance PostgreSQL prouvée après recréation du pod ; alerte de supervision réellement déclenchée puis résolue ; déploiement d'infrastructure rejouable en une commande auto-vérifiée ; sauvegarde produite et restauration vérifiée sans toucher la base principale ; chaque phase validée explicitement et documentée.

2.7 Hors périmètre et évolution du périmètre

Exclusions volontaires (décisions documentées — ADR, `docs/RISKS_AND_TECHNICAL_DEBT.md`) : cloud et Terraform ; registre d'images et déploiement continu distant ; authentification ; Alembic ; Alertmanager et notifications ; Helm ; React ; haute disponibilité.

Évolution du périmètre — l'histoire réelle. Le projet est parti d'un MVP volontairement réduit (application + Compose + 7 tests), cadré par un document initial fixant six phases prévisionnelles et une règle : rien n'entre dans une phase sans décision explicite. Deux extensions ont été décidées en cours de route, tracées dans le dépôt :

- **La phase 6 est devenue une phase produit** : le tableau de bord unique ne permettait pas une démonstration opérateur crédible → console multipage, Command Center par incident, runbooks managés, règles de domaine durcies, campagne de tests portée de 8 à 38 tests.
- **Ansible a été ajouté en fin de projet** : en confrontant le projet aux critères exacts du REAC, j'ai constaté que mon déploiement k3d, documenté mais **manuel**, ne prouvait pas la compétence obligatoire d'automatisation. J'ai fermé cet écart par un périmètre ciblé — automatiser le déploiement existant, sans rien redéfinir (ADR 029, § 5.5, § 6).

Cette progression itérative est une caractéristique du projet : chaque phase validée fige un socle, et une relecture du référentiel a déclenché une correction de périmètre au bon moment.

3. Spécifications techniques

3.1 Architecture logique

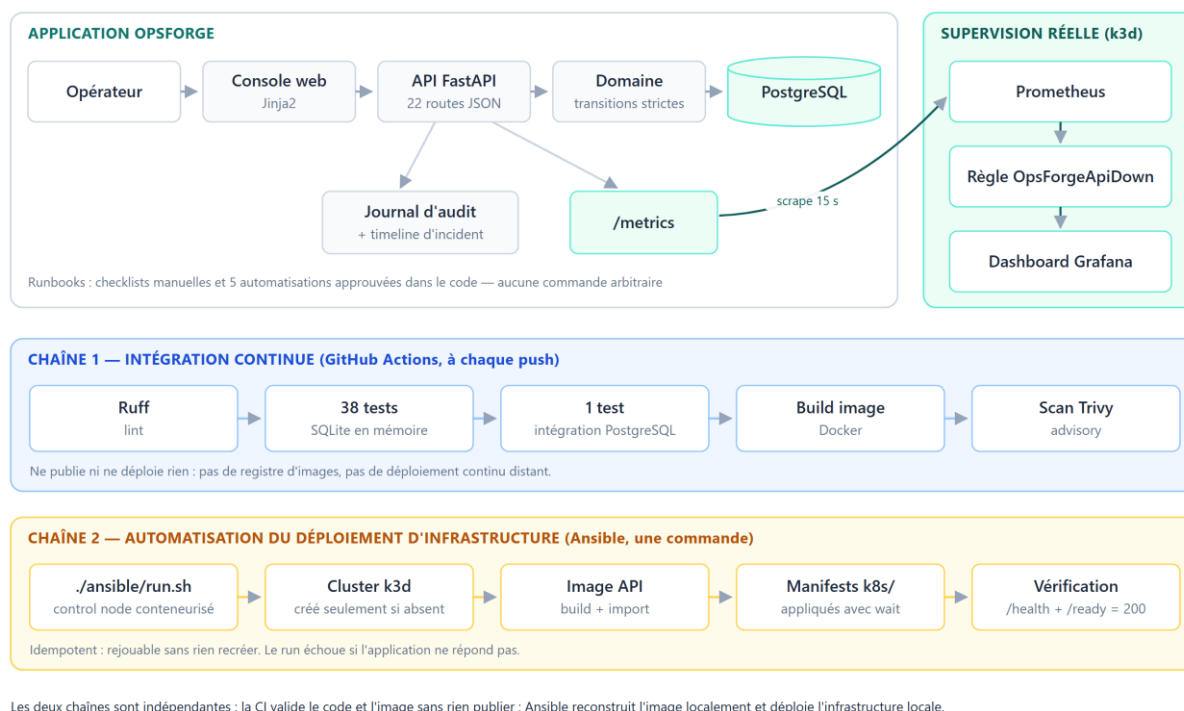


Figure 3 — Architecture logique : l'application et sa supervision réelle (haut), puis les deux chaînes d'outillage (bas). La CI valide le code et l'image sans rien publier ; Ansible reconstruit l'image localement et déploie l'infrastructure.

Les deux chaînes du bas sont **indépendantes** : la CI valide le code et l'image mais ne publie ni ne déploie rien (il n'y a pas de registre) ; Ansible reconstruit l'image localement et déploie l'infrastructure locale.

3.2 Topologie de déploiement

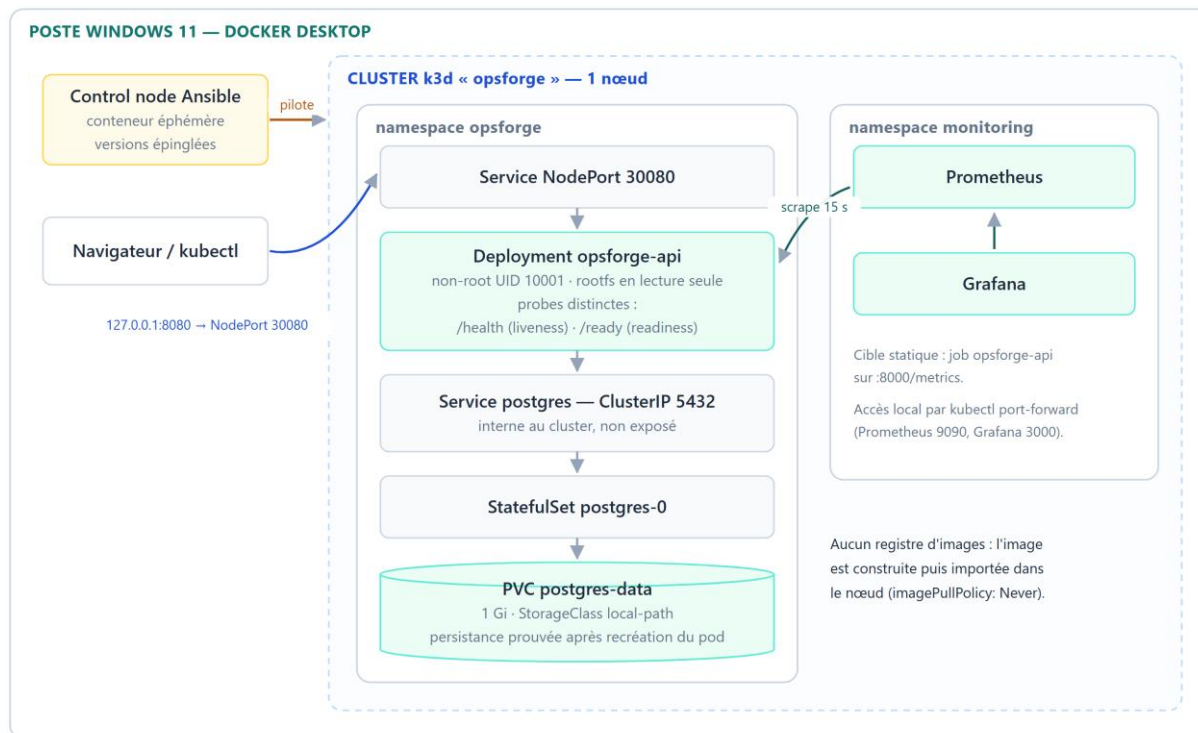


Figure 4 — Topologie déployée : tout tient sur un poste Windows 11 avec Docker Desktop. Le control node Ansible est un conteneur éphémère qui pilote le cluster k3d ; l'API est exposée sur 127.0.0.1:8080 via le NodePort 30080.

L'accès à la supervision se fait par kubectl port-forward (Prometheus 9090, Grafana 3000) — un choix documenté qui évite d'exposer davantage un environnement local.

3.3 Le domaine applicatif

Six objets : Service, Alert, Incident, Runbook, RunbookExecution, AuditLog. Les règles sont appliquées **côté serveur** et testées négativement :

- transitions strictement en avant (alerte new → acknowledged → resolved, résolution directe possible ; incident open → investigating → resolved, sans réouverture) — une transition invalide renvoie HTTP 409 ;
- une alerte n'a qu'un **seul incident actif** (le doublon renvoie 409 avec l'identifiant de l'incident existant) ; l'incident hérite du service de son alerte source ; résoudre l'incident ne résout pas l'alerte ;
- **aucune exécution arbitraire** : 5 clés d'automatisation approuvées dans le code, clé inconnue → 422 ; les runbooks définis dans le code sont « managés » (lecture seule, PATCH → 409) ; un runbook manuel ne peut pas être déclaré réussi avec une checklist incomplète ;
- toute mutation significative et toute tentative d'exécution — y compris les échecs contrôlés — produisent une entrée d'audit.

Structure du code : main.py (démarrage, /health, /ready, /metrics, middleware de métriques), api.py (22 routes JSON et gardes), web.py (8 sections de console, 18 routes HTML), domain.py (transitions), runbooks.py (liste approuvée et moteur), models.py / schemas.py, seed.py (scénario de démonstration idempotent), migrations.py (pont additif de schéma au démarrage).

OpsForge

Incident operations

OPÉRATIONS

Vue d'ensemble

Alertes

Incidents

RESSOURCES

Services

Runbooks

PREUVES

Activité

Monitoring

Aide

Dyllan

Opérateur local

OPSFORGE

Incident #1

Déclarer un incident

Incidents > #1

Incident #1

Critique

En investigation

Sauvegarde de production en échec

La sauvegarde quotidienne de démonstration doit être diagnostiquée avant confirmation du retour à la normale.

Résoudre

Contexte opérationnel

Les informations nécessaires pour comprendre la prise en charge.

SERVICE

Backup Service

production - état de démonstration

RESPONSABLE

DY / Dyllan

DÉCLARÉ

12/08/2026 10:55 UTC

DERNIÈRE MISE À JOUR

12/08/2026 11:00 UTC

ALERTE SOURCE

#1 - Échec de la sauvegarde quotidienne

La sauvegarde planifiée n'a pas produit d'archive valide. Une investigation opérateur est requise.

Modifier le responsable

Dyllan

Mettre à jour

Runbooks compatibles

Procédures contrôlées disponibles dans le contexte de cet incident.

Catalogue

Clôturer l'incident

Automatisé

Moyen

À utiliser uniquement après validation du retour à la normale.

1. Vérifier le diagnostic

2. Confirmer le retour à la normale

Voir la procédure

Exécuter

Contrôler la sauvegarde de démonstration

Automatisé

Faible

Simule un contrôle d'archive sans lancer de commande système.

Diagnostiquer un échec de sauvegarde

Manuel

Faible

Checklist manuelle pour qualifier l'incident de sauvegarde de démonstration.

Générer le rapport d'incident

Automatisé

Faible

Produit un résumé textuel contrôlé de l'incident courant.

Simuler le redémarrage d'un service

Automatisé

Moyen

Enregistre une simulation de redémarrage, sans action sur l'hôte.

Vérifier l'état d'un service

Automatisé

Faible

Lit l'état de démonstration actuellement enregistré pour un service.

Historique des exécutions

Résultats et erreurs produits par les runbooks.

Générer le rapport d'incident

Succès

Incident #1 - Sauvegarde de production en échec. Sévérité=critical, statut=investigating, responsable=Dyllan.

Dyllan - 12/08/2026 11:30 UTC

Chronologie

Source : journal d'audit OpsForge.

Runbook exécuté sur l'incident

Dyllan

12/08/2026 11:30 UTC

Incident #1 - Sauvegarde de production en échec. Sévérité=critical, statut=investigating, responsable=Dyllan.

Statut de l'incident modifié

Dyllan

12/08/2026 11:30 UTC

Incident déclaré

Dyllan

12/08/2026 11:30 UTC

DERNIER RÉSULTAT

Générer le rapport d'incident

Incident #1 - Sauvegarde de production en échec. Sévérité=critical, statut=investigating, responsable=Dyllan.

Succès

Figure 5 — Le Command Center d'un incident : contexte opérationnel, alerte source, runbooks compatibles (manuel ou automatisé, niveau de risque), chronologie issue du journal d'audit et historique des exécutions.

Figure 6 — Le journal d'audit : chaque entrée porte l'acteur, l'action, l'objet lié et l'horodatage UTC. On y lit la chaîne complète d'une prise en charge — alerte créée, incident déclaré, statut modifié, runbook exécuté — les exécutions de runbook étant auditées qu'elles réussissent ou non.

3.4 Conteneurs : de Compose à Kubernetes

- **Image API** : base python:3.12-slim épinglée par digest, utilisateur non-root UID 10001, HEALTHCHECK intégré (§ 5.2).
- **Compose** (développement, tests, sauvegardes) : PostgreSQL avec healthcheck pg_isready et API démarrée seulement quand la base est saine (depends_on: service_healthy) ; base publiée sur 127.0.0.1 uniquement ; tests/ monté en lecture seule.
- **Kubernetes** (déploiement orchestré local) : mêmes conteneurs, contraintes renforcées — init container wait-for-postgres, probes distinctes (/ready = base joignable, /health = processus vivant), securityContext complet, requests et limits (§ 5.3).
- **Mise à jour** : rebuild → k3d image import → rollout ; pas de registre (imagePullPolicy: Never, ADR 015).

3.5 Stockage et persistance (prouvée)

PostgreSQL utilise un **StatefulSet** (identité stable postgres-0, attachement stable au stockage) avec un **PVC de 1 Gi** en StorageClass local-path : le stockage est externalisé du cycle de vie du conteneur et porté par l'hôte. État constaté sur le cluster déployé par Ansible (12/08/2026, extrait de evidence/kubernetes_state.txt) :

pod/opsforge-api-68cbc5b54b-wrdqg	1/1	Running			
pod/postgres-0	1/1	Running			
service/opsforge-api	NodePort	8000:30080/TCP			
persistentvolumeclaim/postgres-data	Bound	1Gi	RWO	local-path	

Preuve de persistance (rejouée le 12/08/2026, `evidence/pvc_persistence.txt`) : un marqueur est inséré, le pod est détruit, le StatefulSet le recrée — l'UID change, la donnée survit :

```
pod UID avant : 554c3801-0b0e-42b7-8bf2-99ced94fd488
INSERT INTO persistence_check VALUES ('dossier-v2-persisted');
pod "postgres-0" deleted -> le StatefulSet recree le pod
pod UID apres : e035ce38-526f-4b3b-9ef5-91aadb3c09f1 (pod reellement nouveau)
SELECT marker : dossier-v2-persisted (donnee retrouvée)
```

Limite : `local-path` est local au nœud — la persistance couvre la recréation du pod, pas la destruction du cluster ; c'est le rôle des sauvegardes (§ 3.7).

3.6 Supervision : réel contre simulé

Le point d'honnêteté central du projet. Prometheus supervise **OpsForge lui-même** (l'API déployée dans k3d). Les statuts métier des services du catalogue sont des données de démonstration saisies dans l'application, et la console l'affiche explicitement.

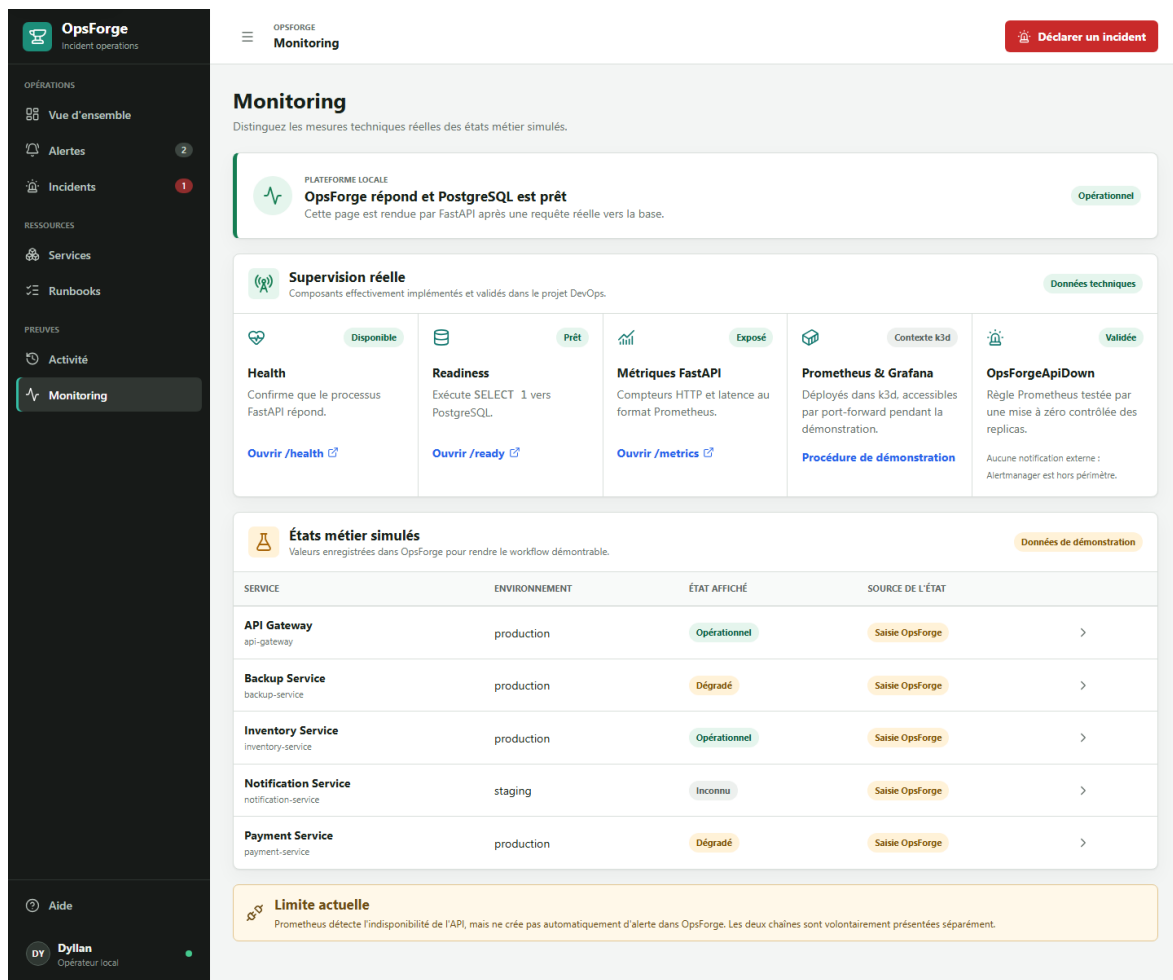


Figure 7 — La page Monitoring sépare la « Supervision réelle » (health, readiness, métriques, Prometheus/Grafana, règle testée) des « États métier simulés » (source : saisie OpsForge), et affiche la limite : les alertes Prometheus ne sont pas ingérées dans OpsForge.

La chaîne réelle : middleware FastAPI → `/metrics` → scrape Prometheus toutes les 15 s (cible statique, job `opsforge-api`) → dashboard Grafana provisionné par ConfigMaps (5 panneaux : disponibilité, volume, codes, latence p95, routes) → règle `OpsForgeApiDown` (`up == 0` pendant 30 s, calibrée sur l'intervalle de scrape).

Panne provoquée et alerte réelle — cycle complet journalisé (12/08/2026,
evidence/prometheus_alert_cycle.txt) :

```
$ kubectl -n opsforge scale deployment/opsforge-api --replicas=0
t+20s : up=0   OpsForgeApiDown=inactive
t+40s : up=0   OpsForgeApiDown=pending
t+70s : up=0   OpsForgeApiDown=firing      >>> FIRING observe <<<
$ kubectl -n opsforge scale deployment/opsforge-api --replicas=1
t+20s : up=1   OpsForgeApiDown=firing
t+40s : up=1   OpsForgeApiDown=inactive    >>> retabli, alerte resolue <<<
```

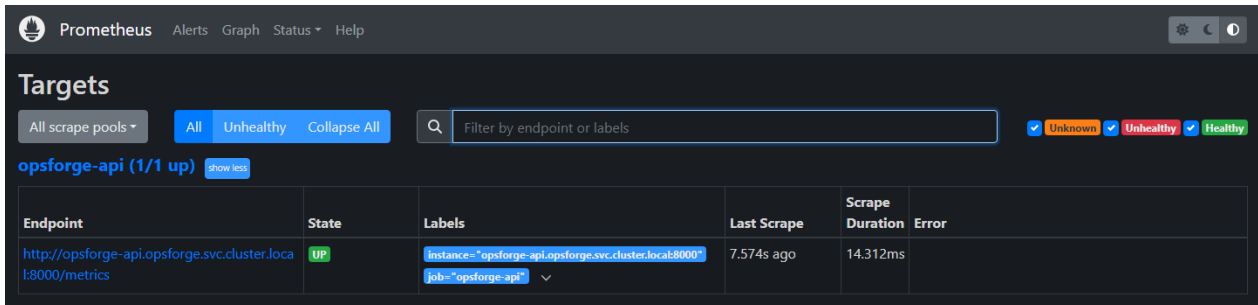


Figure 8 — Prometheus scrape réellement l'API : la cible opsforge-api (1/1 up) pointe sur /metrics du Service Kubernetes, état UP.

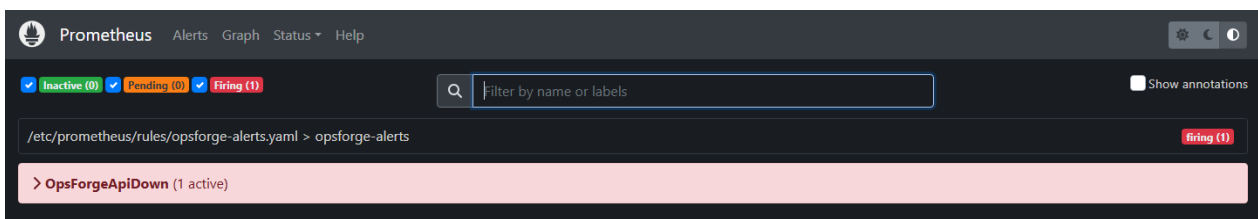


Figure 9 — La même instance pendant la panne provoquée : la règle OpsForgeApiDown est passée en firing (1).

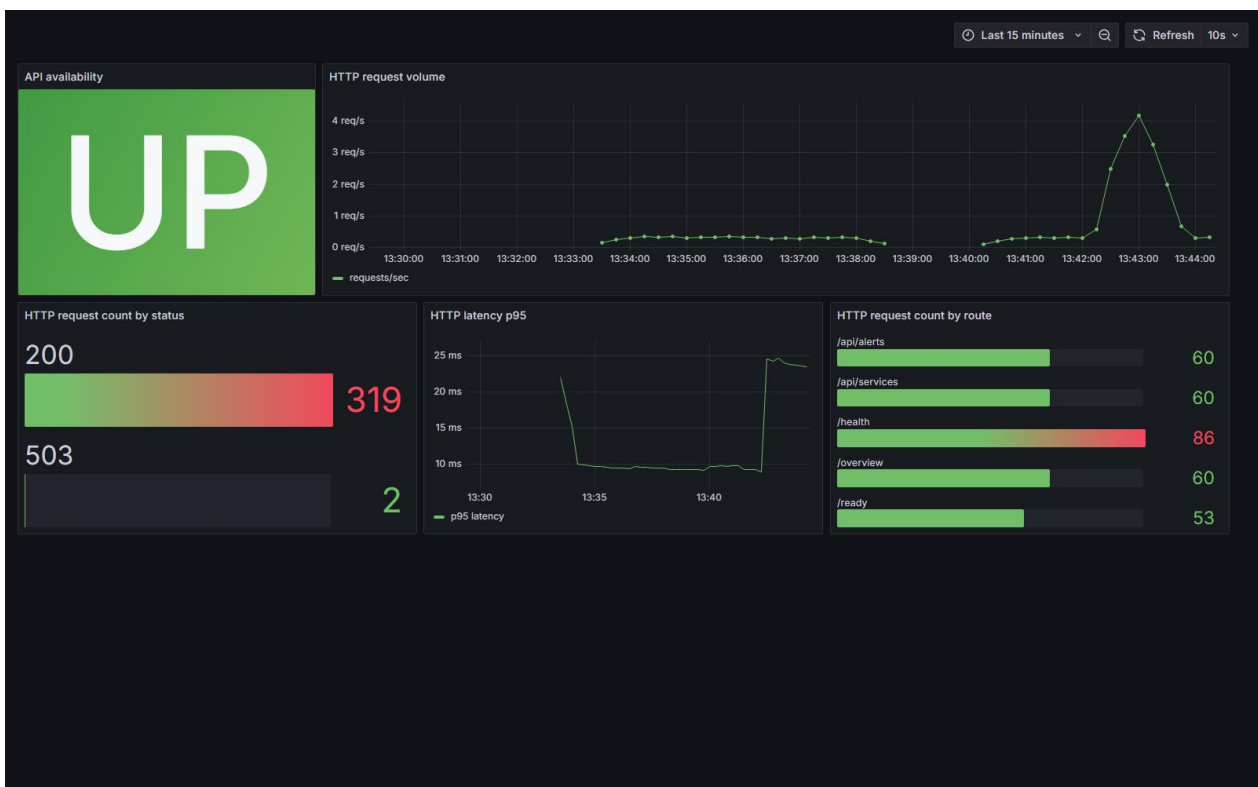


Figure 10 — Le dashboard « OpsForge Monitoring » pendant la session de preuve : disponibilité UP, volume de requêtes (le creux correspond à la panne provoquée), 319 réponses 200 et 2 réponses 503 au redémarrage, latence p95 et répartition par route.

Limites : pas d'Alertmanager (l'alerte est visible dans Prometheus, elle n'est pas routée) ; stockage Prometheus et Grafana éphémère ; métriques techniques HTTP, pas métier.

3.7 Sauvegarde et restauration

Deux scripts PowerShell ciblent le PostgreSQL de Compose : `backup.ps1` (`pg_dump --format=custom` exécuté dans le conteneur, archive copiée vers `backups/`, refus d'une archive vide) et `restore.ps1` (validation `pg_restore --list`, puis **par défaut** restauration dans une base temporaire de vérification ; la base principale exige `-MainDatabase` et la saisie exacte de `RESTORE`). Preuve rejouée le 12/08/2026 (`evidence/backup_restore.txt`) :

```
Backup created: backups\opsforge_backup_20260812_133513.dump (24 543 octets)
Restore verified in temporary database 'opsforge_restore_verify' (6 public tables).
```

Limite : sauvegarde locale, non planifiée, non chiffrée, sans rotation — un mécanisme démontrable, pas une stratégie d'entreprise.

3.8 Sécurité

En couches, toutes documentées : (1) **par conception** — aucune exécution de commande arbitraire, liste d'automatisations approuvées, absence de subprocess et d'eval vérifiée ; (2) **conteneur** — non-root UID 10001, digest épinglé ; (3) **Kubernetes** — securityContext complet vérifié en conditions réelles (§ 5.3) ; (4) **secrets** — `.env` et `secret.local.yaml` ignorés par Git, Secret généré à la volée par Ansible ; (5) **chaîne d'approvisionnement** — scan Trivy **advisory** à chaque build : le dernier scan de l'image épinglée rapportait 19 HIGH et 3 CRITICAL d'origine Debian, sans correctif disponible. Une CI verte ne signifie donc pas « image sans vulnérabilité », et c'est assumé (ADR 013, § 5.4).

3.9 Environnements et versions

Composant	Version
Application	Python 3.12 (digest épinglé), FastAPI / SQLAlchemy 2.x, OpsForge v0.2.0
Base de données	PostgreSQL 16 (<code>postgres:16-alpine</code>) partout
Cluster	k3d v5.9.0 / k3s v1.35.x, kubectl client v1.34
Supervision	Prometheus v2.55.1, Grafana 11.3.1
Control node Ansible	ansible-core 2.17.14, kubernetes.core 6.5.0, client Python kubernetes 36.0.3, kubectl v1.31.5, k3d v5.9.0, CLI Docker 27.5.1 — versions validées, épinglées dans <code>ansible/Dockerfile</code>
Intégration continue	GitHub Actions (<code>checkout@v5</code> , <code>setup-python@v6</code> , <code>trivy-action@v0.36.0</code>)
Poste de travail	Windows 11 + Docker Desktop

Note : le kubectl du control node (v1.31.5) est plus ancien que le k3s du cluster (v1.35.x), au-delà de la fenêtre officielle de ± 1 version mineure ; il est fonctionnel sur les opérations utilisées et validé tel quel — l'alignement est listé en évolution (§ 8).

4. Démarche de travail et outils

4.1 Le découpage en phases et leur état de validation

Phase	Contenu	Validation
1 — MVP local	Application + Compose + 7 tests	16/06/2026
2 — Intégration continue	GitHub Actions : tests, build, scan Trivy (run vert)	18/06/2026
3 — Sauvegarde et sécurité	Sauvegarde/restauration, stratégie de secrets	06/07/2026
4 — Kubernetes	Cluster k3d, PostgreSQL + PVC, API, preuve de persistance	09/07/2026
5 — Supervision	/metrics, Prometheus, Grafana, alerte réellement déclenchée	14/07/2026
6 — Produit opérateur et preuves	Console multipage, domaine durci, durcissement Kubernetes, Ansible (CP n° 2), puis revue manuelle : deux défauts réels corrigés et re-testés (38 tests)	19/08/2026

Chaque phase a un périmètre écrit, une **Definition of Done** vérifiable, un fichier de preuve daté (docs/PHASE<i>_VERIFICATION.md) et une validation explicite. Un protocole écrit en encadre l'avant et l'après : départ propre, périmètre figé — « aucune idée nouvelle n'entre silencieusement dans la phase en cours » — preuves consignées avant validation. Les choix importants sont consignés dans **29 décisions d'architecture**, qui servent aussi de préparation à l'oral.

4.2 Workflow Git

Le workflow a évolué avec le projet : commits directs sur main pour les phases 1 à 5 (un commit de validation par phase), première branche dédiée pour le candidat produit de la phase 6, puis — pour l'audit final et l'ajout d'Ansible — un vrai cycle par branches et **Pull Requests avec commits de merge**, qui préserve l'historique des branches.

Une branche d'expérimentation a servi de terrain de revue puis n'a **jamais été fusionnée** : une branche d'intégration propre a été ré-implémentée en six commits revus, intégrant les corrections identifiées (dont une vraie régression d'interface et la restauration du signal d'échec Trivy). Le même cycle a servi jusqu'au bout : les deux défauts de la revue manuelle finale ont été corrigés en branches dédiées depuis le candidat gelé, revalidés puis fusionnés (PR #4 et #5), la clôture étant enregistrée par une PR documentaire (PR #6). Le détail commit par commit est en annexe A.

4.3 Outils

Python 3.12 / FastAPI / SQLAlchemy / Pydantic / Jinja2 · pytest (SQLite en mémoire et intégration PostgreSQL) · Ruff · Docker et Docker Compose · GitHub Actions · Trivy · k3d/k3s · kubect1 · Ansible (kubernetes.core) · prometheus-client, Prometheus, Grafana · pg_dump / pg_restore en PowerShell · Git et GitHub.

4.4 Conduite du projet et usage de l'IA

OpsForge est un **projet individuel** : pas de client, pas d'équipe — je tiens tous les rôles, et chaque validation de phase est ma décision, tracée dans le dépôt. Dans ce cadre, j'ai utilisé des assistants d'IA comme outils d'aide à la conception, à l'implémentation et à la revue, selon des règles écrites dans le dépôt (docs/ENGINEERING_CHARTER.md).

Je suis resté responsable du cadrage, des choix techniques, des arbitrages et de la validation, et les changements importants ont été vérifiés et testés avant intégration. Ce processus explique le cycle de revue visible dans l'historique Git — « proposition → implémentation → revue → correction → validation » ; dans ce dossier, « revue » désigne cette revue outillée, conduite sous ma responsabilité.

5. Réalisations significatives

Chaque réalisation suit le même fil : **besoin** → **extrait utile** → **choix** → **preuve** → **limite**. Les extraits proviennent du dépôt au commit final a9ec694, condensés pour la lecture (les coupures sont signalées par # [...]) ; les fichiers complets sont dans Git.

5.1 Instrumentation et alerte de supervision (CP n° 10)

Besoin. Superviser réellement OpsForge : des métriques produites par l'application elle-même, et une alerte qui se déclenche vraiment.

```
HTTP_REQUESTS_TOTAL = Counter(
    "opsforge_http_requests_total",
    "Total HTTP requests handled by OpsForge.",
    ["method", "route", "status_code"],
)
# [...] middleware : chronometre chaque requete et alimente compteur + histogramme
labels = {"method": request.method, "route": _route_label(request),
         "status_code": str(response.status_code)}
```

```
- alert: OpsForgeApiDown
  expr: up{job="opsforge-api"} == 0
  for: 30s
  labels: {severity: critical, service: opsforge-api}
```

Choix. Le label route utilise le *template* FastAPI (/api/services/{service_id}), pas l'URL réelle : sans cela, chaque identifiant créerait une série Prometheus distincte — explosion de cardinalité (ADR 017). /metrics est exclu de son propre comptage. La règle s'appuie sur la métrique up du scrape : elle détecte l'injoignabilité quelle qu'en soit la cause, là où une métrique applicative disparaîtrait avec l'application ; for: 30s est calibré sur le scrape de 15 s, de sorte qu'un raté isolé ne déclenche pas l'alerte.

Preuve. Cible UP, dashboard alimenté, et cycle inactive → pending → firing → résolu observé en direct (figures 7 à 10, § 3.6). **Limite.** Pas de routage de notification, faute d'Alertmanager.

5.2 Image Docker durcie (CP n° 7)

Besoin. Une image reproductible qui ne tourne jamais en root, dans Compose comme dans Kubernetes.

```
FROM python:3.12-slim@sha256:c3d81d25b3154142b0b42eb1e61300024426268edeb5b5a26dd7ddf64d9daf28
# [...]
RUN useradd --create-home --shell /usr/sbin/nologin --uid 10001 opsforge \
    && chown opsforge:opsforge /app
USER opsforge
# [...]
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD ["python", "-c", "from urllib.request import urlopen;
    urlopen('http://127.0.0.1:8000/health', timeout=3)"]
```

Choix. Digest épinglé : deux builds espacés donnent la même base, et le scan porte sur une image identifiée (ADR 024). L'UID 10001 est réutilisé tel quel dans le securityContext Kubernetes. Le healthcheck est écrit en Python pur — aucune dépendance ajoutée pour la sonde. **Preuve.** Image construite à chaque commit en CI ; conteneur healthy en Compose ; la même image tourne sous rootfs en lecture seule dans k3d. **Limite.** Les 19 HIGH et 3 CRITICAL Debian résiduels du scan restent visibles et sans correctif disponible — assumés.

5.3 Workloads Kubernetes durcis et stockage persistant (CP n° 7)

Besoin. Déployer avec le principe du moindre privilège, des démarrages ordonnés, et un stockage qui survit aux pods.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  runAsGroup: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: {drop: ["ALL"]}
  seccompProfile: {type: RuntimeDefault}
# [...]
readinessProbe: {httpGet: {path: /ready, port: http}}
livenessProbe: {httpGet: {path: /health, port: http}}
```

Choix. Les deux probes sont **différentes**, et c'est le point clé : `/ready` (un `SELECT 1` sur PostgreSQL) retire l'API du Service quand la base est injoignable ; `/health` ne redémarre le conteneur que si le processus ne répond plus. Utiliser `/ready` en liveness provoquerait des redémarrages en boucle pendant une panne de base. Un init container `pg_isready` sérialise le démarrage — l'équivalent du `depends_on` de Compose. PostgreSQL est un **StatefulSet** adossé au PVC (§ 3.5) ; seul `/tmp` (`emptyDir`) est inscriptible.

Preuve. Vérifié sur cluster réel : pods 1/1 Ready, pages en 200 sous rootfs en lecture seule, écriture dans `/app` refusée (audit du 07/08/2026) ; état re-constaté et persistance re-prouvée le 12/08/2026 (§ 3.5). **Limite.** Une réplique de chaque workload — la haute disponibilité n'est pas l'objectif du périmètre.

5.4 Pipeline d'intégration continue (chaîne 1)

Besoin. Vérifier chaque commit : qualité, comportement du domaine, compatibilité avec le moteur de base réel, image, vulnérabilités.

```
- name: Lint with Ruff
  run: ruff check .
- name: Run SQLite unit tests
  run: pytest tests/test_app.py
- name: Run PostgreSQL integration test
  env: {DATABASE_URL: postgresql+psycopg://opsforge:opsforge@127.0.0.1:5432/opsforge}
  run: pytest tests/postgres_integration.py
- name: Build Docker image
  run: docker build -t "${IMAGE_NAME}:${IMAGE_TAG}" .
- name: Run Trivy image scan (advisory)
  uses: aquasecurity/trivy-action@v0.36.0
  continue-on-error: true
  with: {scan-type: image, image-ref: "opsforge-api:${{ github.sha }}",
    exit-code: "1", severity: "HIGH,CRITICAL"} # [...]
```

Choix. L'ordre est voulu : lint (échec rapide) → 38 tests SQLite (quelques secondes) → test d'intégration contre un conteneur `postgres:16-alpine` démarré par le job. Ce test crée une base au nom unique, rejoue le flux complet, puis la supprime : la CI ne peut pas polluer une base de démonstration. La subtilité Trivy : `exit-code: "1"` garde le signal visible quand des HIGH ou CRITICAL existent, tandis que `continue-on-error: true` maintient le job vert — une politique advisory explicite (ADR 013), transformable en portail bloquant par une décision documentée.

Preuve. Run du candidat final a9ec694 vérifié le 19/08/2026 via l'API GitHub
(evidence/github_actions_run.txt) :

```
run 32197168814 - CI - branch phase6-operator-ux - sha a9ec694 - conclusion: success

Lint with Ruff ..... success      Build Docker image ..... success
Run SQLite unit tests ..... success  Run Trivy image scan (advisory) success (*)
Run PostgreSQL integration test success
```

(*) L'API GitHub aplatit l'issue d'une étape *continue-on-error* ; l'interface web, elle, affiche l'état *advisory* de l'étape.

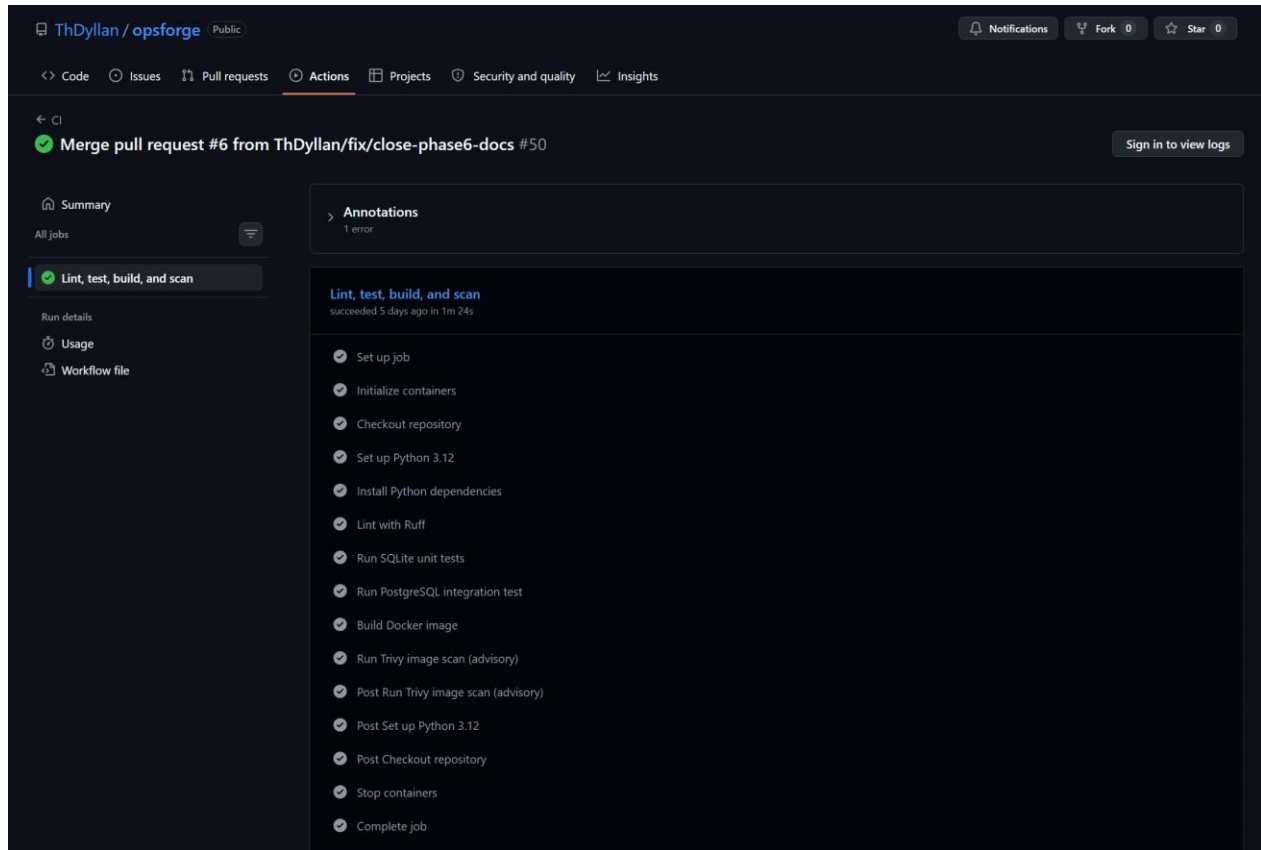


Figure 11 — La même exécution vue dans l'interface : le job est **vert** et porte pourtant **une annotation d'erreur**. C'est toute la politique *advisory* — l'étape Trivy signale son `exit code 1` (annotation *failure*, vérifiée par l'API), *continue-on-error* empêche ce signal de bloquer la livraison, et l'API de haut niveau l'aplatit en *success*. Le titre du run identifie le candidat final : *Merge pull request #6*.

Limite. La CI ne publie pas l'image et ne déploie rien : c'est de l'intégration continue, pas du déploiement continu distant.

5.5 Automatisation du déploiement d'infrastructure avec Ansible (CP n° 2, chaîne 2)

Besoin. Remplacer la séquence manuelle documentée (`k3d cluster create` puis une série de `kubectl apply`) par une commande unique, idempotente et auto-vérifiée.

Pourquoi Ansible plutôt que Terraform. Les deux sont cités par le référentiel et légitimes — et Terraform n'est pas « réservé au cloud ». Terraform est déclaratif et à état : on décrit un état cible qu'il réconcilie via des providers. Ma tâche est une **orchestration procédurale multi-outils locale** — préparer le control node, créer le cluster, construire et importer l'image, appliquer des manifests dans un ordre imposé, attendre, vérifier en HTTP : la forme d'exécution qu'Ansible exprime naturellement, la collection `kubernetes.core` apportant le déclaratif sur les ressources Kubernetes. Terraform deviendrait pertinent pour un provisioning cloud, direction de la CP n° 4, non réalisée (ADR 029).

```
# role cluster - creation idempotente
- name: Create the k3d cluster
  ansible.builtin.command: >-
    k3d cluster create {{ cluster_name }} --servers 1
    --api-port 127.0.0.1:{{ kubeapi_host_port }}
    --port "{{ api_host_port }}:{{ node_port }}@server:0" --wait
  when: cluster_name not in (k3d_clusters.stdout | from_json | map(attribute='name') | list)

# role kubernetes_resources - Secret genere, deploiement ordonne et attendu
- name: Create the PostgreSQL Secret from variables
  kubernetes.core.k8s:
    definition:
      kind: Secret
      stringData: {POSTGRES_USER: "{{ db_user }}", POSTGRES_PASSWORD: "{{ db_password }}" } # [...]
- name: Deploy PostgreSQL and wait until it is ready
  kubernetes.core.k8s: {src: "{{ manifests_path }}/{{ item }}", wait: true} # [...]

# role verify - le deploiement se prouve lui-meme
- name: Check /ready through the NodePort (proves PostgreSQL connectivity)
  ansible.builtin.uri:
    url: "http://host.docker.internal:{{ api_host_port }}/ready"
    retries: 12
  until: ready.status is defined and ready.status == 200
```

Choix. Le cluster n'est créé que s'il n'existe pas, ce qui rend le playbook rejouable sans danger. **Aucun manifest de Secret n'est versionné** : il est généré au déploiement depuis des variables (défaut local non sensible, surchargeable par `-e` ou `ansible-vault`). Chaque étage est appliqué avec `wait: true` — PostgreSQL prêt, puis API prête, puis supervision — et non par un apply aveugle. Enfin, le run ne réussit **que si l'application répond** : `/ready` prouve la chaîne complète jusqu'à la base. Le tout s'exécute depuis un **control node conteneurisé** (`./ansible/run.sh`) aux versions épinglées — indispensable sous Windows, et né d'une vraie situation de recherche (§ 6).

Preuve. Rejouée intégralement le 12/08/2026 sur un cluster isolé (`evidence/ansible_*.txt`) :

```
Deploiement complet      : PLAY RECAP  ok=22  changed=9  failed=0
                          "/health -> 200 {'status': 'ok', 'service': 'opsforge'}"
                          "/ready  -> 200 {'status': 'ready', 'service': 'opsforge'}"
Second run (idempotence) : PLAY RECAP  ok=21  changed=2  failed=0
Teardown                 : PLAY RECAP  ok=3    changed=1  failed=0
```

Limites. Cible k3d locale uniquement ; le control node conteneurisé est la seule méthode validée ; `validate_certs: false` est limité à la connexion de contrôle locale vers le cluster éphémère.

5.6 Sauvegarde et restauration sûres

Besoin. Prouver qu'une sauvegarde existe **et** qu'elle est restaurable, sans jamais risquer la base par une fausse manœuvre.

```
if ($MainDatabase) {  
    $confirmation = Read-Host "This will replace objects in database '$dbName'. Type RESTORE to  
    continue"  
    if ($confirmation -cne "RESTORE") { Write-Host "Restore cancelled." ; return }  
}
```

Choix. Format custom, inspectable par `pg_restore --list`; restauration **par défaut dans une base temporaire** de vérification, avec comptage des tables puis nettoyage ; la base principale exige le drapeau -MainDatabase **et** la saisie exacte de RESTORE, sensible à la casse — une touche Entrée pressée trop vite annule l'opération ; l'API est arrêtée pendant une restauration réelle, puis redémarrée. **Preuve.** Rejouée le 12/08/2026 : archive de 24 543 octets créée, puis restauration vérifiée sur 6 tables dans `opsforge_restore_verify` (§ 3.7). **Limite.** Locale, non planifiée, non chiffrée.

6. Situation de travail ayant nécessité une recherche

Le fichier `ansible.cfg` silencieusement ignoré dans le control node conteneurisé

Le problème. Situation survenue lors de l'intégration d'Ansible (10 et 11 août 2026), traçable dans le dépôt (commits 9d04fc4, 0b21505, 732d6fa, e808ccf). Ansible ne s'exécutant pas nativement sous Windows, j'avais choisi un control node conteneurisé : une image Docker embarquant Ansible et les CLI, le dépôt monté en `/work`. Première exécution du playbook dans ce conteneur :

```
skipping: no hosts matched
```

Aucun hôte trouvé — alors que l'inventaire (`localhost ansible_connection=local`) existait, déclaré dans `ansible.cfg` juste à côté du playbook.

Le diagnostic. Hypothèses testées dans l'ordre. Inventaire mal écrit ? Non : il fonctionnait passé à la main. Mauvais répertoire de travail ? Non : le `WORKDIR` était correct. Le fichier `ansible.cfg` est-il seulement lu ? `ansible --version` affichait `config file = None` : Ansible ignorait silencieusement la configuration, donc l'inventaire qu'elle déclare.

La recherche. La documentation officielle d'Ansible décrit un comportement de sécurité précis : **un `ansible.cfg` situé dans un répertoire courant inscriptible par tous (*world-writable*) est refusé**, pour empêcher l'injection d'une configuration malveillante dans un répertoire partagé ; un chemin désigné via `ANSIBLE_CONFIG` reste honoré. Or c'était exactement mon contexte, sans que je l'aie provoqué : un dépôt Windows monté par bind-mount Docker apparaît *world-writable* côté Linux. Le comportement d'Ansible était correct — c'est mon environnement qui le déclenchait.

Les options et le choix. Trois pistes : `chmod` du répertoire à chaque run (fragile) ; tout passer en ligne de commande sans `ansible.cfg` (ce qui disperse la configuration) ; fixer `ANSIBLE_CONFIG` dans l'image — la solution prévue par l'outil. J'ai retenu la troisième, doublée d'une ceinture de sécurité : `ENV ANSIBLE_CONFIG=/work/ansible/ansible.cfg` cuit dans l'image, **plus** un `-i inventory.ini` explicite dans `run.sh`.

Le second enseignement — le plus important. En corrigeant, j'ai découvert plus embarrassant que le bug : mes validations initiales avaient été faites avec des commandes ajustées à la main pendant le débogage, et le `run.sh` documenté ne portait pas ces réglages — ni l'inventaire explicite, ni `--add-host=host.docker.internal:host-gateway`, ni `MSYS_NO_PATHCONV=1`. Quiconque aurait exécuté le `./run.sh` documenté — le jury, par exemple — aurait reproduit l'échec initial. Le commit de correction le dit sans détour : « *run.sh did not carry the settings the successful runs actually used* ». J'ai donc :

- aligné `run.sh` sur le chemin réellement validé ;
- **re-testé l'intégralité du parcours en n'utilisant que `./run.sh`** sur un cluster isolé (déploiement complet, second run idempotent, teardown), puis épinglé les versions exactes du control node validé pour garder la validation reproductible ;
- retiré de la documentation une affirmation que je ne pouvais pas prouver : le commit initial prétendait que le playbook « tourne aussi directement sur un hôte Linux/WSL », un chemin jamais validé qui n'aurait pas fonctionné tel quel (le `kubeconfig` est réécrit vers `host.docker.internal`, un nom fourni par le conteneur). Le control node est devenu **la seule méthode supportée et validée**.

Ce que j'en retiens. Un comportement de sécurité d'un outil peut n'apparaître que dans un contexte d'exécution particulier : le diagnostic passe par la documentation, pas par des essais au hasard. Et surtout, **l'artefact documenté doit être exactement celui qui a été validé** — retirer de sa propre documentation une affirmation non prouvée est une correction au même titre qu'un correctif de code.

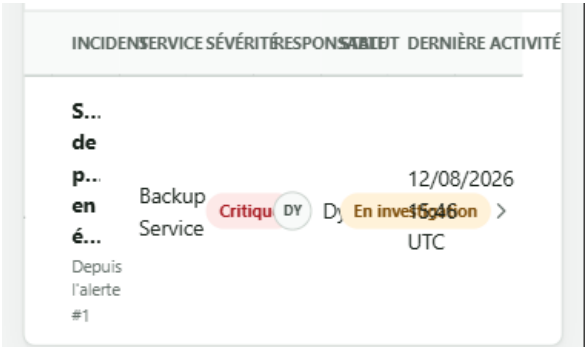
7. Synthèse des validations

Domaine	Preuve principale	Résultat
Application et tests	38 tests SQLite + 1 test d'intégration PostgreSQL (base éphémère)	Verts en local et en CI
Intégration continue	Run GitHub Actions du candidat final a9ec694	success (toutes étapes) — vérifié le 19/08/2026
Conteneurs et Kubernetes	Pods 1/1 Running, PVC Bound, durcissement vérifié sous rootfs en lecture seule	Constaté le 07/08 et re-constaté le 12/08/2026
Persistance	Donnée survivant à la destruction puis à la recréation du pod PostgreSQL (UID différent)	Prouvée en phase 4, re-prouvée le 12/08/2026
Supervision	Cible UP, dashboard alimenté, cycle d'alerte inactive → firing → résolu	Validée en phase 5, rejouée en direct le 12/08/2026
Automatisation d'infrastructure	Déploiement complet en une commande, idempotence, teardown, /health et /ready en 200	Validée aux commits Ansible, rejouée le 12/08/2026 (ok=22/9, ok=21/2)
Sauvegarde	Archive produite et restauration vérifiée en base temporaire (6 tables)	Validée en phase 3, rejouée le 12/08/2026

Les preuves du 12/08/2026 — logs bruts et captures — sont versionnées sous `deliverables/evidence/` et `deliverables/assets/`; l'historique détaillé phase par phase reste dans `docs/PHASE<i>_VERIFICATION.md` (chronologie en annexe A).

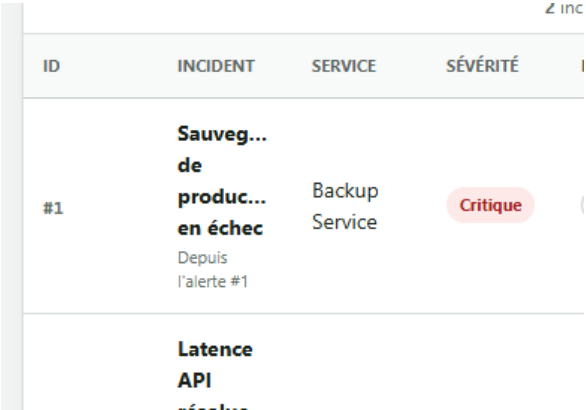
Clôture de la phase 6 : ce que la revue humaine a trouvé

La revue manuelle — parcours desktop, workflow opérateur complet, comportement responsive — a été déroulée du 12 au 19/08/2026, guidée par un runbook dédié. Elle a détecté **deux défauts réels, invisibles des 35 tests automatisés alors en place** : une erreur HTTP 422 sur les formulaires de filtres lorsque aucun service n'était sélectionné, et une table Incidents illisible sur mobile. Les deux ont été corrigés en branches dédiées, couverts par des tests de non-régression qui ont porté la suite à **38 tests**, puis revalidés humainement sur le candidat fusionné ; la phase 6 a alors été **explicitement validée le 19/08/2026** (PR #4 à #6, candidat final a9ec694).



INCIDENT	SERVICE	SÉVÉRITÉ	RESPONSABLE	DERNIÈRE ACTIVITÉ
Sauvegarde de production en échec	Backup Service	Critique	DY	En investigation
12/08/2026 15:46 UTC				

Figure 12 — Avant : à 390 px, les en-têtes de la table Incidents se chevauchent et le titre de l'incident est tronqué à quelques caractères.



ID	INCIDENT	SERVICE	SÉVÉRITÉ
#1	Sauvegarde de production en échec	Backup Service	Critique

Figure 13 — Après correction : colonnes lisibles, la table défilant dans son propre cadre.

Cette séquence est un enseignement en soi : les deux défauts passaient sous le radar de tests qui vérifient des réponses HTTP et des structures de page, mais ne simulent ni la soumission réelle d'un formulaire ni un rendu à 390 px. **Une procédure de test manuelle documentée n'est pas un reliquat d'un projet non automatisé : c'est la couche qui attrape ce que l'automatisation ne voit pas.**

8. Limites assumées et évolutions

Toutes ces limites sont documentées dans le dépôt (`docs/RISKS_AND_TECHNICAL_DEBT.md`, ADR). Ce sont des choix de périmètre, pas des fonctions prétendues :

Domaine	Limite	Évolution naturelle
Infrastructure	k3d mono-nœud local, pas de cloud ; stockage local - path local au nœud, non distribué	Kubernetes managé + Terraform (CP n° 4) ; stockage réseau
Livraison	Pas de registre, pas de CD distant ; import k3d (manuel ou par Ansible)	Registre + déploiement déclenché par la CI
Application	Pas d'authentification (acteurs déclaratifs) ; unicité d'incident actif garantie par l'application (409), pas par contrainte en base	Authentification ; contrainte PostgreSQL partielle
Schéma	<code>create_all()</code> et pont additif, pas d'historique de migrations	Alembic
Supervision	Pas d'Alertmanager ni de notifications ; stockage éphémère ; Grafana <code>admin/admin</code> en local ; accès par port-forward ; métriques techniques ; états métier simulés	Alertmanager, persistance, métriques métier, ingestion des alertes dans OpsForge
Sécurité	Trivy advisory (19 HIGH / 3 CRITICAL sans correctif, visibles, non bloquants) ; pas de TLS	Seuil de blocage explicite ; TLS
Sauvegardes	Locales, non planifiées, non chiffrées, sans rotation	Planification, chiffrement, externalisation
Outillage	Écart de versions entre le <code>kubectl</code> du control node (v1.31) et k3s (v1.35)	Alignement des versions épinglées
Contexte	Projet individuel : le critère relationnel « échanges avec les développeurs » (CP n° 10) est porté par des décisions tracées, pas par une équipe	—

Conclusion

Ce que le projet démontre. Les trois compétences obligatoires, chacune avec une preuve distincte, rejouable et rejouée : une infrastructure locale complète déployée et vérifiée par Ansible en une commande idempotente ; des conteneurs durcis et orchestrés dans deux environnements, avec une persistance prouvée plutôt que supposée ; une supervision réelle dont l'alerte s'est déclenchée puis résolue lors d'une panne provoquée. Autour : 38 tests unitaires et un test d'intégration PostgreSQL, une CI avec scan de vulnérabilités, des sauvegardes restaurables, 29 décisions d'architecture documentées.

Mes satisfactions. La méthode — six phases finies, validées et datées une à une — qui a permis au projet de survivre sans dégât à un changement de poste de travail. La correction de trajectoire de la fin de projet : confronter le travail aux critères exacts du référentiel, constater qu'un déploiement documenté mais manuel ne prouvait pas la compétence d'automatisation, et fermer l'écart proprement, avant l'examen. Et une exigence tenue de bout en bout : distinguer partout ce qui a été testé de ce qui a seulement été écrit.

Mes difficultés. Le débogage du control node Ansible (§ 6), le plus formateur — avec sa leçon : l'artefact documenté doit être exactement celui qui a été validé. La revue visuelle finale, qui n'a pas pu être automatisée : déroulée manuellement, elle a révélé deux défauts réels que les 35 tests d'alors ne voyaient pas — preuve que le contrôle humain reste une étape à part entière. Et, en continu, tenir le périmètre : dire non à tout ce qui aurait grossi le projet sans le rendre plus défendable.

Le projet est gelé au commit `a9ec694` : c'est cet état, reproductible et documenté, que je présente au jury.

Annexe A — Chronologie détaillée du projet

Date	Événement	Commits
16/06/2026	Validation du MVP (7 tests ; travail pré-Git)	—
17/06/2026	Premier commit : application MVP et workflow CI	ad9b9df
18/06/2026	Phase 2 validée sur un run GitHub Actions vert ; gouvernance (index, risques, protocole)	9e0666f, 917378b
06/07/2026	Phase 3 : sauvegarde et restauration, workflow Git	a317969, 34f785b
07-09/07/2026	Phase 4 : k3d + PostgreSQL/PVC (4A, vérifiée le 07), API (4B), preuve de persistance, validation	c386c84, 2772b50
09-14/07/2026	Phase 5 : métriques (5A), Prometheus (5B), Grafana (5C), alerte et panne provoquée (5D), validation	84fb228 → 23194f0, 9bcb271
15-17/07/2026	Phase 6 : candidat produit opérateur — débuté sur main, poursuivi sur la première branche dédiée phase6-operator-ux	83469cb → 230d07a
07/08/2026	Audit : branche d'expérimentation revue (jamais fusionnée, conservée comme trace), ré-implémentation propre en 6 commits sur integration/phase6-audit (35 tests, durcissement Kubernetes vérifié sur cluster réel)	fb4d77c → fd3dca9
10-11/08/2026	Ansible (CP n° 2) : implémentation, corrections de la situation de recherche (§ 6), épinglage, honnêteté documentaire	9d04fc4, 0b21505, 732d6fa, e808ccf
11/08/2026	Intégration finale par Pull Requests avec commits de merge : PR #1 (audit), PR #2 (Ansible), PR #3 (synchronisation documentaire) → candidat gelé	489552f, 0becdf8, 8ab0f70
12/08/2026	Production des preuves du présent dossier : re-déploiement Ansible complet, idempotence et teardown, persistance re-prouvée, cycle d'alerte joué en direct, sauvegarde et restauration jouées, CI du candidat vérifiée, captures d'écran	branche jury/dossier-fil-rouge
12-19/08/2026	Revue manuelle finale (runbook dédié) : deux défauts réels détectés — erreur 422 des formulaires de filtres et table Incidents illisible en mobile — corrigés en branches dédiées avec tests de non-régression (35 → 38 tests), plus le favicon ; revalidation humaine sur le candidat corrigé	PR #4 3fc7707, PR #5 f6e4a79
19/08/2026	Phase 6 validée explicitement ; clôture documentaire (roadmap, vérification, risques) → candidat final	PR #6 a9ec694

Le candidat d'examen phase6-operator-ux @ a9ec694 totalise **45 commits**, dont 6 commits de merge. La branche main porte l'état des phases 1 à 6 initiales.

Annexe B — Inventaire des preuves

B.1 Captures et schémas (versionnés sous deliverables/assets/)

Fichier	Contenu	Usage
diagrams/schema_architecture.png	Architecture logique et les deux chaînes d'outillage	Figure 2
diagrams/schema_topologie.png	Topologie de déploiement sur le poste	Figure 4
screenshots/01_overview.png	Vue d'ensemble de la console	Figure 1
screenshots/02_alerts.png	File d'alertes : les trois états, les filtres, le lien vers l'incident ouvert	Figure 2 (*)
screenshots/03_incident_command_center.png	Command Center : contexte, runbooks, timeline, exécutions	Figure 5
screenshots/04_activity.png	Journal d'audit : acteur, action, objet lié, horodatage	Figure 6 (*)
screenshots/05_monitoring.png	Page Monitoring : réel contre simulé	Figure 7
screenshots/06_prometheus_targets_up.png	Cible Prometheus opsforge-api (1/1 up)	Figure 8 (*)
screenshots/07_prometheus_alert_firing.png	OpsForgeApiDown en FIRING	Figure 9 (*)
screenshots/08_grafana_dashboard.png	Dashboard « OpsForge Monitoring » alimenté	Figure 10
screenshots/09_command_center_mobile.png	Command Center en mobile (~390 px)	Réserve orale
screenshots/10_incidents_mobile.png	Table Incidents en mobile après correction	Figure 13 (*)
screenshots/12_incidents_mobile_avant.png	Table Incidents en mobile à l'état défectueux, constaté pendant la revue	Figure 12 (*)
screenshots/11_github_actions_run.png	Interface GitHub Actions : job du run 32197168814 (candidat final) avec son annotation	Figure 11 (*)

(*) Figures recadrées sur la zone utile ; les fichiers listés sont les captures complètes, non retouchées, et les versions recadrées sont versionnées sous `deliverables/assets/figures/`. La capture GitHub Actions provient de la page publique du dépôt, sans session authentifiée — le bouton « Sign in to view logs » qui y figure en atteste.

B.2 Journaux d'exécution bruts (versionnés sous deliverables/evidence/)

Fichier	Contenu
ansible_fresh_deploy.txt	Déploiement complet par Ansible : ok=22 changed=9, /health et /ready en 200
ansible_second_run.txt	Second run : idempotence ok=21 changed=2
ansible_teardown.txt	Suppression du cluster de test
kubernetes_state.txt	État constaté : pods, services, PVC, nœud
pvc_persistence.txt	Preuve de persistance : marqueur et UID du pod avant/après
prometheus_alert_cycle.txt	Cycle d'alerte complet horodaté et payload de l'alerte
backup_restore.txt	Archive produite et restauration vérifiée
github_actions_run.txt	Run CI du candidat final a9ec694, étape par étape

Les environnements de production de ces preuves sont tracés : les captures d'interface ont été prises sur un environnement Docker Compose isolé et éphémère, et les preuves Kubernetes, Ansible et supervision sur un cluster k3d jetable, distinct du cluster de travail.

Annexe C — Reproduire les preuves

Chaque preuve de ce dossier est rejouable. Les commandes ci-dessous sont celles qui ont produit les journaux versionnés sous `deliverables/evidence/`. Prérequis : le dépôt cloné et Docker Desktop.

C.1 Application et tests

```
docker compose up --build -d          # console : http://localhost:8000/overview
curl.exe http://localhost:8000/health # {"status": "ok", ...}
curl.exe http://localhost:8000/ready  # {"status": "ready", ...} - SELECT 1 sur PostgreSQL
docker compose exec api pytest        # 38 tests SQLite
docker compose exec api pytest tests/postgres_integration.py # integration PostgreSQL
```

C.2 Déploiement de l'infrastructure par Ansible (CP n° 2)

À lancer **sur un cluster jetable**, pour ne pas agir sur un cluster existant :

```
# deployment complet auto-verifie (ok=22 changed=9) ; rejoue -> idempotence (ok=21 changed=2)
./ansible/run.sh deploy.yml -e cluster_name=opsforge-ansible-test \
  -e api_host_port=8090 -e kubeapi_host_port=6446
./ansible/run.sh teardown.yml -e cluster_name=opsforge-ansible-test # suppression du cluster
```

Attention : `./run.sh` sans paramètre vise le cluster `opsforge` et agirait sur un cluster existant. Le control node conteneurisé est la seule méthode validée (§ 6).

C.3 Persistance du stockage (CP n° 7)

```
kubectl -n opsforge exec postgres-0 -- psql -U opsforge -d opsforge \
  -c "CREATE TABLE persistence_check(m text); INSERT INTO persistence_check VALUES('preuve');"
kubectl -n opsforge delete pod postgres-0
kubectl -n opsforge rollout status statefulset/postgres --timeout=180s
kubectl -n opsforge exec postgres-0 -- psql -U opsforge -d opsforge \
  -tAc "SELECT m FROM persistence_check;" # marqueur retrouve = persistance prouvée
```

C.4 Supervision et déclenchement réel de l'alerte (CP n° 10)

```
kubectl -n monitoring port-forward svc/prometheus 9090:9090 # /targets et /alerts
kubectl -n monitoring port-forward svc/grafana 3000:3000 # dashboard
kubectl -n opsforge scale deployment/opsforge-api --replicas=0 # panne provoquée : up == 0
# OpsForgeApiDown : inactive -> pending -> firing (au bout de 30 s)
kubectl -n opsforge scale deployment/opsforge-api --replicas=1 # A RESTAURER systématiquement
```

C.5 Sauvegarde et restauration

```
.\scripts\backup.ps1
.\scripts\restore.ps1 -BackupFile .\backups\opsforge_backup_AAAAMMJJ_HHMMSS.dump
```

Sans le drapeau `-MainDatabase`, la restauration se fait dans une base temporaire de vérification : la base principale n'est jamais touchée (§ 5.6). Enfin, le dépôt étant public, le run d'intégration continue du candidat final se consulte sans compte à l'adresse github.com/ThDyllan/opsforge/actions/runs/32197168814 (figure 11).

Annexe D — Glossaire

Terme	Définition dans le contexte du projet
ADR	<i>Architecture Decision Record</i> : décision consignée (contexte, décision, raison, conséquences) — 29 dans docs/DECISIONS.md
Advisory (scan)	Résultats visibles mais non bloquants pour le pipeline
Command Center	Vue de travail d'un incident : contexte, responsable, transitions, runbooks, exécutions, timeline
Control node	Machine — ici un conteneur — depuis laquelle Ansible s'exécute
CP / REAC	Compétence professionnelle / Référentiel Emploi Activités Compétences du titre
DoD	<i>Definition of Done</i> : conditions vérifiables de fin d'une phase
Idempotence	Une automatisation rejouée converge sans rien recréer ni casser
k3d	Cluster Kubernetes k3s exécuté dans des conteneurs Docker
Liveness / readiness	« Le processus vit-il ? » (redémarrage) / « Peut-il servir ? » (retrait du Service)
NodePort	Service Kubernetes exposant un port du nœud (30080 → 127.0.0.1:8080)
PVC	<i>PersistentVolumeClaim</i> : demande de stockage persistant (1 Gi, local-path)
Runbook	Procédure : checklist manuelle ou action automatisée approuvée dans le code
Scrape	Collecte périodique des métriques par Prometheus (toutes les 15 s)
StatefulSet	Contrôleur pour workloads à état : identité et stockage stables (PostgreSQL)